Name	Student Number	GitHub ID
Rivin Sharma	24295598	rivinsharma
Wenkow Sun	24168584	wenkow2k515
Shravan Suresh Kumar	24975803	ShravansKumar
Kennice Leong Jing Lin	24271935	kenniceleongg
Nathan Lance Macaspac Buluran	23402727	naetnaetnaet

GitHub Link: https://github.com/wenkow2k515/CITS5501-3501-Group-Project.git

Planning and organization

1. Where is your group's Collaboration & Workflow Plan maintained? Is it in the Git repository, or elsewhere?

Our group has decided to maintain our Collaboration and Workflow Plan in our shared Git repository, much like all the documents we'll be creating for this project. The main reason and advantage that Git provides is a single location for the shared document, meaning that the team won't have to search other avenues for this document. Furthermore, the version history aspect of Git makes it a very appealing option which made us favour it the most. However, it is important to acknowledge a main trade-off of this decision is that Git doesn't provide any convenient way for real-time collaboration that something like a shared Word or Google doc would have.

2. What tool or approach are you using to track assigned tasks and completion? Why did you choose it? Has anyone in the group used it before?

We've chosen to track assigned tasks and their completion within our repository under the issues tab. Much like aforementioned, we're aiming to keep our Git repository as our single source of information, mainly to reduce the time and effort that will be taken to search through other sources or locations. Furthermore, many members are familiar with Git and Github through the unit Agile Web Development and are comfortable and understand how to use it.

Github is also a standard practice in the professional industry, so wherever possible we feel it's important to practice efficient and right use of this tool.

Language and tooling For this project, you may choose to implement your application in either Java or Python.

3. Which language has your group chosen? Have any members of the group not used it

– and if so, how will they get familiar with it? What factors influenced your choice of implementation language? Rank the top three in order of importance.
– Familiarity with the language
– Team members' experience
– Library support
– Ease of debugging
– Performance
– Maintainability
– Assessment considerations
– Career value
– Other (explain) What advantages do you expect your chosen language to give your team? What disadvantages or risks do you expect? (Every choice has costs.) If you had to implement this project in the other language, what do you think would be most difficult? What might be easier?

Upon researching the use of Java and Python in creating a web scraper, ultimately the team decided to go with Python. Not only is it the language which our team has the most experience and familiarity with, but also the library support which Python has. Python contains well-documented libraries which would assist in every corner of this project.

However, it's important to note that this decision comes with disadvantages and risks. For example, by choosing Python we're deprioritising performance. Although the performance won't be degraded significantly since scraping would be I/O bound, Python is generally slower than Java due to it being CPU-heavy. Furthermore, Python is dynamically typed, meaning that bugs which Java's compiler would catch will only surface at runtime for python.

We could presume that by choosing Java instead, there would be stricter compile-time checks which could lead to us catching bugs and faults earlier. However, due to our team's knowledge and comfortability with using Python, the trade-off of the longer onboarding time using Java was not justifiable leading to our eventual decision of using Python.

4. What build/dependency management tool will you use (e.g. uv or poetry for Python; Gradle or Maven for Java; something else)? Has anyone in the group used it before – or will this be the first time setting up a project with it? Technical risks

We've decided to use uv for dependency and environment management. Based on the brief we were given, uv is already installed on the Ubuntu 22.04 marking environment, so we know without fail that this will work when our code is tested. Compared to other options, like poetry, uv handles dependency resolution, virtual environments and locking in one tool without needing separate config files to remain in sync.

Nobody in this group has used uv before, though most of the team has used pip and is familiar with the underlying concepts of declaring dependencies, virtual environments etc. so we don't expect there to be a huge learning curve since uv's command set is quite small and well-documented.

5. What parts of a web scraper do you expect could most easily fail, and how?

As the brief has said, the Web can be unpredictable. As a result, the parts of a scraper which are most likely to fail could include:

- HTML Parsing: different documents could use different page templates, so a selector which works on one page may silently fail on another.

- Network requests: Many requests could fail for reasons such as timeouts, DNS failures, or if a file is large enough then the connection could drop mid-download

- Rate limiting: Should requests be made too frequently the server could temporarily block our IP

- Format extractions: PDF files which are scanned images with no extractable text layer could fail, we could also encounter things like corrupted files or Word documents which are saved in the older .doc format as opposed to the .docx

With some of these issues we run into the problem that they would fail silently. The program will still run, however we end up with wrong, missing or incomplete data. Therefore, our testing strategy will need to verify the correctness of our program, not just check that the code runs without throwing an exception.

6. Which external factors could make your software stop working after it has been completed?
With the Web being unpredictable, there are a few external factors that could affect our software. For example:

- Website redesigning: If there are changes made to the HTML structure or CSS classes, this could break our parsing which would result in silent errors and give us missing or wrong data

- URL changes: If we include any hardcoded links or link-discovery pattern, these would break should documents or pages move to new locations

- Authentication changes: content that was previously open changes due to a login wall, CAPTCHA or paywall being added

- Format changes: As mentioned in question 5, format extractions could be an aspect that causes our software to fail. To build on this, if this format changes to a format we didn't initially build to support, our software could stop working

- The site going down or being decommissioned entirely.

7. Which components of your project do you think will be easiest to test? Which do you expect will be hardest?

We expect that the parts of the application which contain all the deterministic logic and don't depend on the website directly will be the easiest components to test. This may include functions responsible for parsing command line arguments, filtering or searching retrieved documents, validating types of files, and handling locally stored data. This is because these can be tested using fixed inputs with clearly defined expected outputs. We are able to isolate and test these components repeatedly using pytest without the need to access the live website.

We expect that the parts of the application which involve external network behaviour and interaction with the website will be the hardest components to test. Network requests may be affected by rate limiting, timeouts, server errors, broken links and unexpected responses, all of which make tests against the live server very unreliable. To reduce unreliability, we expect to use test techniques such as mocks and controlled test data to simulate these situations. Another area that we expect to be difficult is document extraction. This is because PDF's, Word documents, and RTF files may have different structures, and are also prone to corrupted and malformed states.

8. What assumptions is your software likely to make about the website it scrapes? How could those assumptions be violated?

Assumptions about the structure and behaviour of the website we scrape are inevitable. This could mean we assume that pages keep a reasonably consistent HTML structure, links to policy documents are able to be identified in a predictable way, and URLs that are returned by the website point to valid and accessible resources. Another appropriate assumption is that the server also returns appropriate HTTP responses, and the downloaded documents are in the form of the supported formats such as PDF, Word or RTF.

The assumptions stated above could then be violated if the website changes HTML structure, renames CSS classes/elements, changes URL format, removes documents, or a new page

layout is introduced. A URL is also susceptible to return an error page rather than an expected document, or a file extension may not accurately represent the actual file type. The website may also introduce authentication to access, or a stronger rate limiting in the future.

Where assumptions cannot be confirmed from the specification, will be documented explicitly. The scraper shall be designed such that unexpected inputs are handled gracefully, rather than crashing the entire program.

9. Suppose a serious bug is discovered two days before the deadline. How would your team investigate and respond?

Our team would respond by setting direct priorities. Our first priority would be to ensure that the issue is reproduced consistently while determining the scope of the issue and its severity. We would then create test cases that demonstrate the failure that the issue is causing. Along with this we would inspect relevant logs such as Git history to see recent changes to help identify when and where the bug was introduced.

Once we have narrowed down the cause, the bug would be posted as an issue in Git, and it shall be assigned to one or more members, and the rest shall avoid making unnecessary changes to the affected site. The members would implement the fix on a separate branch and also include a regression test that fails before fix is implemented and passes after the fix is implemented. Then the existing test suite, linting and type checking would be run to ensure there are no additional issues introduced from the fix.

Because of the time constraint, we would prioritise a stable and submission compliant scraper rather than trying to implement new features or large refactors. The fix would go through our normal review process such as any other feature would, before being merged into the branch.

10. What will convince your team that a feature is “finished”?

A feature will be considered finished when its required behaviour has been implemented, relevant tests have been written and passed, linting and type checking pass, documentation has been updated where necessary, and the pull request has been reviewed and approved before merging.

11. If two group members disagree about whether software is “working correctly”, how will you resolve that disagreement? Code quality
We will refer back to the project specification and documented assumptions to decide the expected behaviour. If possible, we will create a reproducible test case and compare the actual result with the expected result. If the specification is still unclear, we will ask the teaching staff for clarification rather than resolving the disagreement by opinion alone .

12. Python groups: Will you use a type checker (e.g. mypy or pyright), and if so, will it run in CI or just periodically/manually? Has anyone used one before? Do you plant to use a linter (e.g. pylint, black)? Java groups: Is your team familiar with any static analysis/linting tools (e.g. Checkstyle, SpotBugs, PMD)? Do you plan to use any?
We plan to use Pyright for static type checking. These checks will initially be run locally and later integrated into our CI workflow so that they are automatically run on pull requests. We also plan to use Ruff for linting and formatting.

13. Suppose a group member's contribution doesn't pass your linting or type checking checks. What will be the consequences? Will merges be blocked if these checks fail, or will the checks be advisory only? Testing framework
If linting or type checking fails, the pull request should not be merged into the main branch. The author should fix the reported issues and rerun the checks before the pull request can be approved and merged.

14. What test framework will you use (e.g. pytest for Python; JUnit 5 for Java; something else)? Has the group used it before, or only heard of it? CI / automation
We plan to use pytest as our main testing framework because it supports unittest, fixtures, parameterised tests and mocking. Which are useful for testing both local logic and simulated network behaviour.

15. Will you set up continuous integration (e.g. GitHub Actions) to run your test suite automatically, or run tests manually before submission? Have any members of your group used continuous integration before? Testing mindset
We plan to use GitHub Actions for continuous integration. Pull requests will automatically run the test suite, linting, and type checking before changes can be merged into the main branch. This helps ensure that the same checks are applied consistently to every contribution.

16. Suppose your scraper is finished and submitted. Now imagine another student is trying to break your software (get it to exhibit failures or unexpected behaviour). What might they try first?

Probably the first thing they'd try is feeding it broken input like dead links, pages where the HTML has changed from what we expected, or files that claim to be a PDF/Word but actually aren't. Another obvious one is hammering the site with requests to see if we handle rate limiting properly, or cutting the connection halfway through a download. On the command-line side, they'd likely try weird or missing arguments, or search queries that return nothing, just to see if the program handles it gracefully instead of crashing.

17. How will you know that your software is producing the correct output, rather than merely some output?

The plan is to build a handful of test cases where we already know what the "correct" answer should be. We grab a real page or document, manually check what should be extracted from it, and then assert our scraper actually returns that. That way we're comparing against a

known-correct value, not just checking that nothing crashed. For cases where we can't pin down an exact expected value in advance (e.g., text pulled from a real PDF), we'll spot-check a sample by hand and also check things like "is the file type right, is the content non-empty, are the expected fields there."

18. What evidence would increase your confidence that your software is reliable?

There are a few things: our tests passing consistently in CI, not just once locally; Pyright and Ruff coming back clean; every PR having actually been reviewed by someone else before merging; and running the scraper against the live site a few times and getting the same correct result each time. We'd also feel more confident if we deliberately tried to break it ourselves and it failed cleanly instead of quietly returning wrong data.

19. At this stage of the project, what do you think will be the biggest challenge in producing reliable software?

Right now the biggest issue is that we're planning this without having actually seen the site yet, so a lot of what we're deciding is based on assumptions rather than the real thing. That makes it hard to know how much effort to put into robustness versus just getting the core features working. On top of that, a lot of the trickiest stuff to test, for instance, network failures, rate limiting, weird documents- isn't something we can easily reproduce reliably, so we'll need to lean on mocks and saved test fixtures rather than testing against the live server every time.

20. What will be your code review process — does someone other than the author review changes before merging, or is this informal?

It's formal because nobody can push straight to main, everything goes through a PR, and it has to be reviewed and approved by someone other than whoever wrote it before it can merge. PRs need a short description of what they do, and commits need proper messages. Merging is also blocked unless CI passes (tests, linting, type checking), regardless of whether it's been reviewed, so a change can't sneak through even if it looks fine on the surface.